Voice Agent MVP: Task & Complete Flow

Assignee	Estimate	Priority	Type
SDE 1	~2 sprints (4 milestones)	P1	Feature / new service

1. Context & goal

We want a voice agent that a user can talk to in real time. The agent listens, understands what was said, decides how to respond with an LLM (optionally calling tools), and speaks the reply back. This task covers the full pipeline from microphone to speaker, plus logging and basic error handling.

Goal: a user opens a web page, clicks **Start Call**, speaks naturally, and hears a relevant spoken reply within **~1.5 s** of finishing their sentence. The conversation keeps context across turns until they click **End Call**.

2. System architecture

Each block below is a separate module behind an interface. Solid arrows are the main data path; dashed arrows are side channels (live captions, logging).

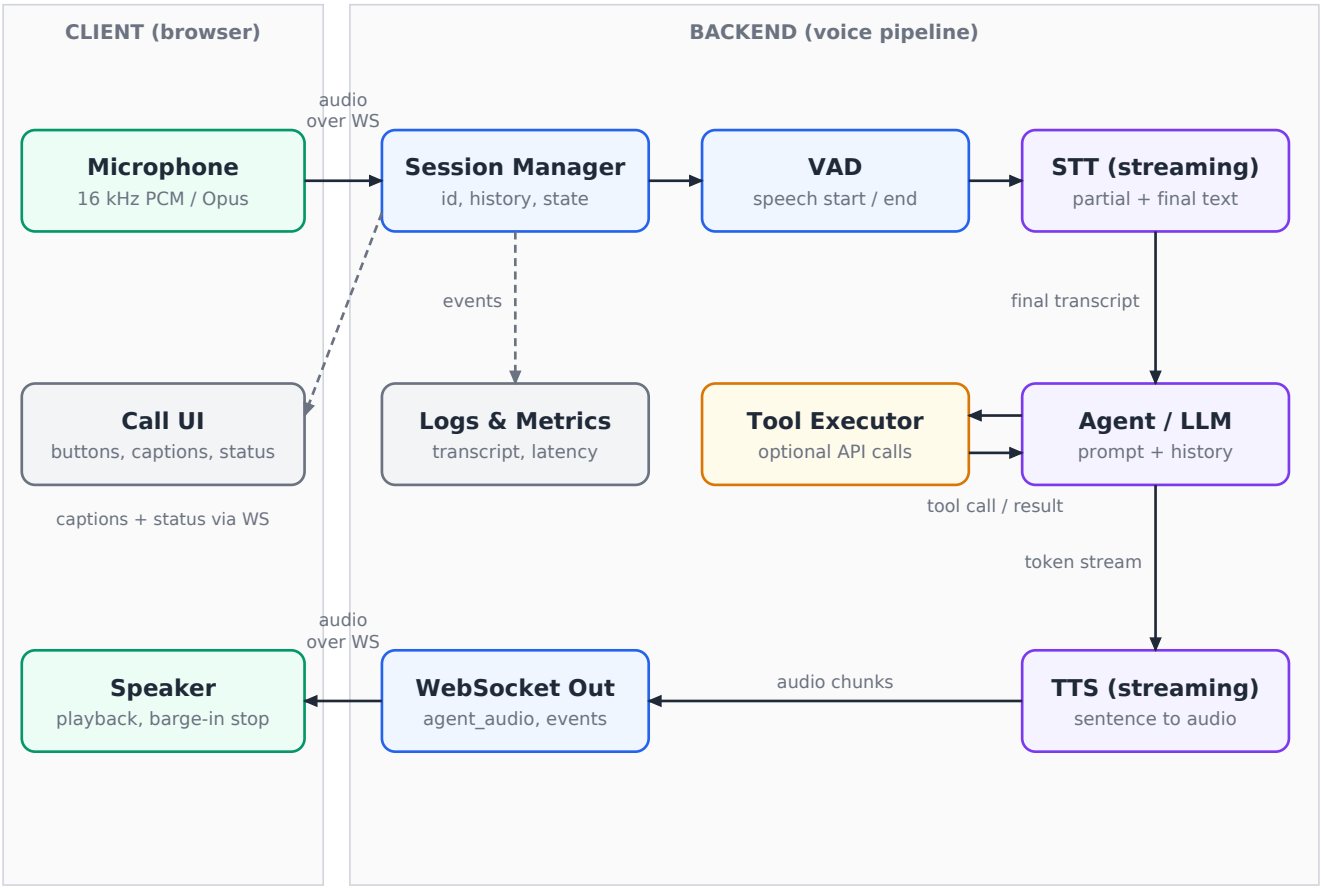


Figure 1. End-to-end data flow from the user's microphone back to their speaker.

Components

Component	Responsibility	Input → Output
Session Manager	Create/end sessions; hold history and state (listening / thinking / speaking); own timeouts.	POST /sessions → session_id
VAD	Detect speech start and end of utterance (500–700 ms silence, configurable). Drop noise-only audio.	audio frames → speech segments
STT (streaming)	Convert speech to text; emit partials for captions and a final transcript per turn.	audio → partial / final text
Agent / LLM	Build prompt (system + history + transcript); stream tokens; decide on tool calls.	text → token stream / tool call
Tool Executor	Run requested tool (sample: get_current_time or mock order status) and return result to LLM.	tool call → JSON result
TTS (streaming)	Synthesize speech sentence by sentence as tokens arrive; support cancel.	text → audio chunks
Client (web)	Mic capture, playback, captions, status, barge-in detection, Start/End buttons.	user ↔ WebSocket
Logs & Metrics	Persist transcript, per-stage timestamps and errors per session.	events → DB rows

3. Complete call flow, step by step

Step	What happens
1. Start call	Client calls `POST /sessions` and receives a `session_id`. It opens `WS /ws/voice?session_id=…`, asks for mic permission, and starts streaming audio. State → LISTENING.
2. Listen	Server runs VAD on each audio frame. When speech starts, frames are forwarded to the streaming STT. Noise-only or very short segments are ignored (no LLM call).
3. Transcribe	STT returns partial transcripts (sent to client as `transcript_partial` for live captions). When VAD detects end of utterance, STT emits the final transcript (`transcript_final`). State → THINKING.
4. Think	Final transcript is appended to history and sent to the LLM with the system prompt. If the LLM requests a tool, the Tool Executor runs it and returns the result; the LLM then writes the final reply. Tokens are streamed.
5. Speak	Tokens are buffered into sentences; each complete sentence goes to TTS immediately (don't wait for the full reply). Audio chunks stream to the client as `agent_audio`; text goes as `agent_text`. State → SPEAKING.
6. Barge-in	If the user speaks while the agent is talking, the client stops playback and sends `interrupt`. Server cancels in-flight LLM and TTS, keeps what was actually said in history, and returns to step 2.
7. Next turn	When the reply finishes playing, state returns to LISTENING and steps 2–6 repeat.
8. End call	Triggered by End Call, 2 min silence, max call duration, or WebSocket drop. Server sends `call_ended`, closes the socket, and persists the transcript and metrics. State → ENDED.

Session state machine

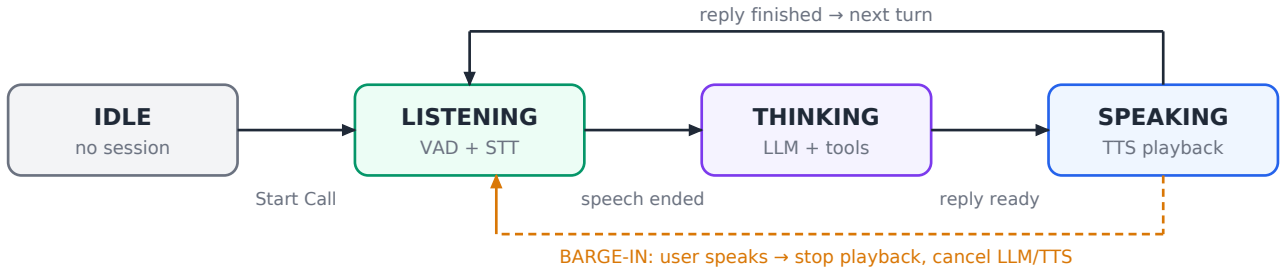
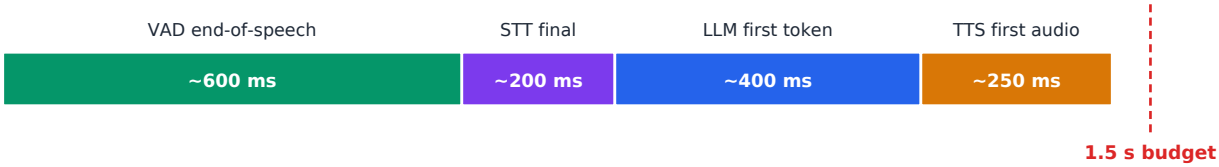


Figure 2. Session states. Any state can move to ENDED. Barge-in jumps from SPEAKING back to LISTENING.

Latency budget



Indicative budget: user stops talking → first agent audio ≈ 1450 ms (includes the VAD silence wait). Measure every stage from day one.

4. API & WebSocket events

Endpoint / event	Direction	Payload (key fields)	Purpose
POST /sessions	client → server	{ } → { session_id }	Create a call session
WS /ws/voice	both	binary audio + JSON events	Real-time channel
audio (binary)	client → server	PCM 16 kHz mono frames	User speech
transcript_partial	server → client	{ text }	Live captions
transcript_final	server → client	{ text, turn_id }	Committed user turn
agent_text	server → client	{ text, turn_id }	Agent caption
agent_audio	server → client	binary audio chunk	Agent speech
state	server → client	{ state }	listening / thinking / speaking
interrupt	client → server	{ turn_id }	Barge-in
error	server → client	{ code, message }	Recoverable error
call_ended	server → client	{ reason }	Session closed
GET /sessions/:id	client → server	→ { transcript, metrics }	Fetch saved session

Data saved per session

- **session:** id, started_at, ended_at, end_reason, config used (providers, prompt version).
- **turns:** turn_id, role (user / agent), text, interrupted (bool), tool calls + results.
- **metrics per turn:** vad_end_ms, stt_final_ms, llm_first_token_ms, tts_first_audio_ms, total_ms.
- **errors:** stage, code, message, timestamp.

5. Scope

In scope

- Backend session manager, WebSocket endpoint, VAD, and pluggable **STT / LLM / TTS adapters** behind interfaces.
- One sample tool to prove the tool-calling path.
- Per-stage latency timing and per-session persistence.
- Simple web page: Start / End Call, mic permission handling, live captions, status indicator, barge-in.
- Config via env/config: system prompt, VAD silence threshold, max call duration, provider keys.

Out of scope

- Telephony (Twilio / SIP), auth, multi-language, voice cloning, horizontal scaling, analytics dashboard.

6. Error handling

Failure	Expected behavior
Mic permission denied	Clear message on UI; no session created.
STT / LLM / TTS timeout or error	Agent says a fallback line ("Sorry, I didn't catch that, could you repeat?"); error logged; call continues.
WebSocket drops	Client shows "Disconnected"; session marked ended; transcript saved.
Empty / noise-only utterance	Ignored; no LLM call.
Tool call fails	LLM receives the error and responds gracefully.
Max duration / silence timeout	Agent says a short goodbye, then call_ended is sent.

7. Acceptance criteria

	Criterion
☐	User completes a 5+ turn conversation and the agent remembers earlier turns.
☐	Median time from end of user speech to first agent audio ≤ 1.5 s (from logs, 20 test turns).
☐	Live captions show partial and final transcripts.
☐	Barge-in works: agent audio stops within ~300 ms of user speaking.
☐	Sample tool is invoked correctly when asked a relevant question.
☐	All failure cases in section 6 behave as described.
☐	Each session's transcript and per-stage latencies are persisted and retrievable by session_id.
☐	Swapping one provider (e.g. TTS) needs only config + a new adapter, no pipeline changes.
☐	Unit tests for state transitions and adapters (mocked); one integration test running a recorded audio file through the full pipeline.
☐	README with setup, env vars, architecture diagram and how to run locally.

8. Milestones

Milestone	When	Outcome	Review
M1 · Design + skeleton	Day 1–3	Design doc (flow, event schema, interfaces); WebSocket echo client ↔ server.	Lead reviews design
M2 · Happy path	Week 1	Mic → STT → LLM → TTS → speaker for one turn (non-streaming OK).	Demo to lead
M3 · Real-time	Week 2 (1st half)	Streaming STT/LLM/TTS, multi-turn history, captions, sentence-level TTS.	Latency check
M4 · Robustness	Week 2 (2nd half)	Barge-in, tool call, error handling, metrics, tests, README, demo.	Final PR review

9. Deliverables

- PR(s) with backend, frontend, tests and README.
- 2–3 min demo recording of a multi-turn call including a barge-in and a tool call.
- Latency report (median + p90 per stage) from 20 test turns.

10. Guidance for the assignee

- Write the design doc and event schema first; get it reviewed before building the pipeline.
- Build non-streaming first, then make each stage streaming one at a time.
- Add latency timing from day one; it is much harder to add later.
- Keep provider code behind interfaces; the pipeline should not know which vendor it calls.
- Pin down audio format and sample rate early (client capture, STT input, TTS output); this is where most time gets lost.
- Raise blockers early in stand-up instead of waiting for the milestone review.